# Web Technology Lab 07

## Multi-Role User System (API + UI + RBAC)

**Name:** Kunal Sharma
**Enrollment No:** CS-2341057
**Section:** 3CSE15 (G1)
**GitHub Link:**
https://github.com/Roshians/Web-Tech-Lab-06

**Instructor:** Mr. Dibyanarayan Hazra

# 1 Overview

The objective of this lab is to develop a comprehensive Fullstack Role-Based Access Control (RBAC) system. Using Node.js and Express for the backend and React for the frontend, we implemented a system that distinguishes between three user roles: **SUPER_ADMIN**, **ADMIN**, and **USER**. The system ensures secure authentication using JWT (JSON Web Tokens) and password hashing via bcrypt.

## 1.1 Roles and Permissions (RBAC Table)

| **Action** | **USER** | **ADMIN** | **SUPER_ADMIN** |
|---|---|---|---|
| Register (Public) | Yes | - | - |
| Login | Yes | Yes | Yes |
| View Own Profile (/users/me) | Yes | Yes | Yes |
| List Users (/users) | No | Yes (USERs only) | Yes (All) |
| Create User (/users) | No | Yes (USER only) | Yes (USER/ADMIN) |
| Change Role (/users/:id/role) | No | No | Yes |
| Delete Users (/users/:id) | No | Yes (USER only) | Yes (Any) |

# 2 API Documentation Table

| **#** | **Method** | **Endpoint** | **Auth** | **Role** | **Req. Body** | **Response** | **Fig ID** |
|---|---|---|---|---|---|---|---|
| 1 | POST | /api/auth/register | No | Public | {name, email, password} | 201 Created | Fig-1 |
| 2 | POST | /api/auth/login | No | Public | {email, password} | 200 Token + User | Fig-2 |
| 3 | GET | /api/users/me | Yes | All | - | 200 Profile | Fig-3 |
| 4 | GET | /api/users | Yes | ADMIN / SUPER | - | 200 User List | Fig-4 |
| 5 | POST | /api/users | Yes | ADMIN / SUPER | {name, email, password, role} | 201 Created | Fig-5 |
| 6 | PATCH | /api/users/:id/role | Yes | SUPER_ADMIN | {role} | 200 Updated | Fig-6 |
| 7 | DELETE | /api/users/:id | Yes | ADMIN / SUPER | - | 200 Deleted | Fig-7 |

# 3 API Screenshots

This section contains screenshots of Postman tests proving the API functionality and role-based access.

# 4 RBAC Proof (Forbidden Examples)

Demonstrating server-side enforcement of role restrictions (403 Forbidden).

# 5 UI Screenshots

Visual proof of the role-based React UI rendering features based on permissions.

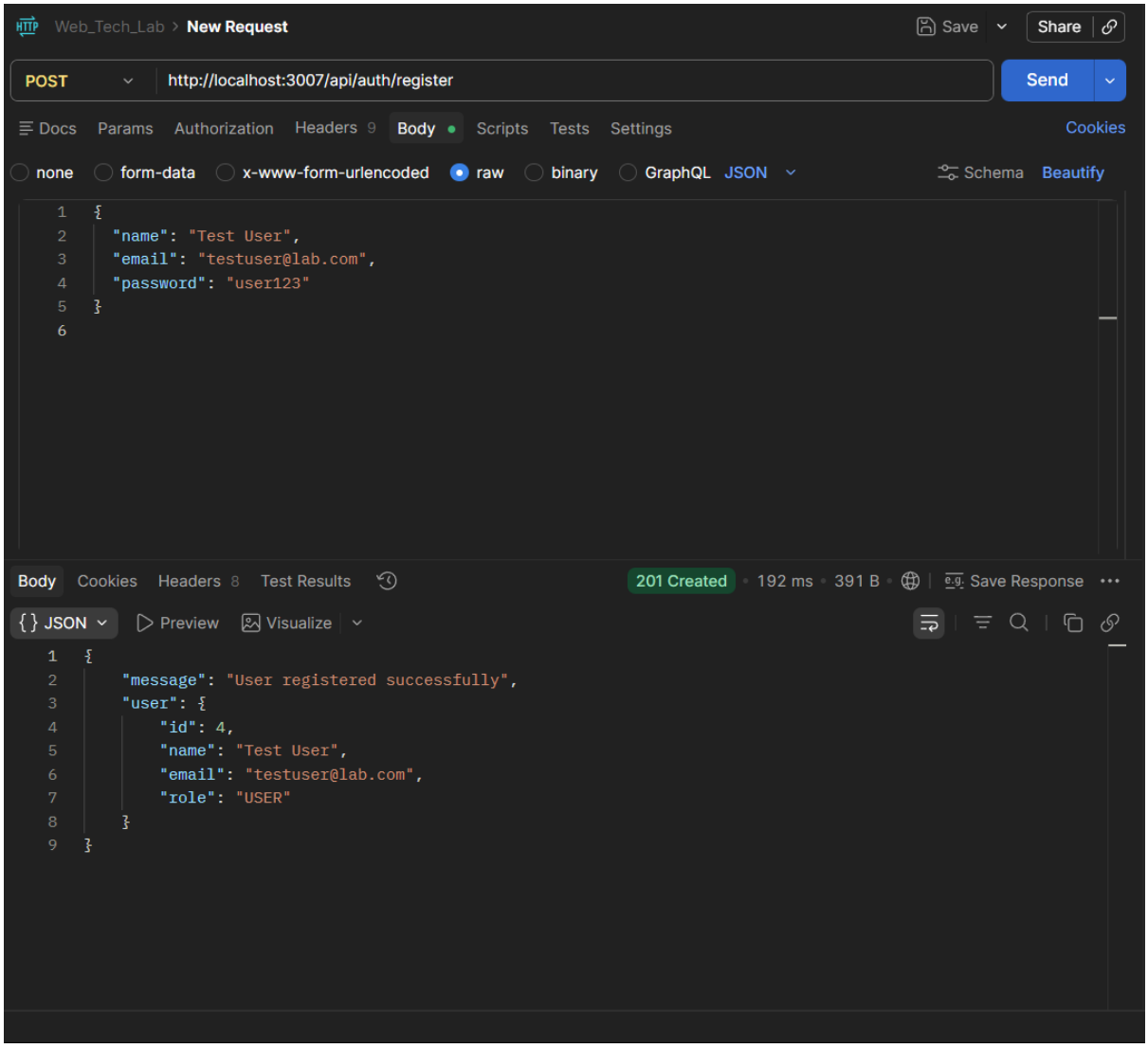


Figure 1: Fig-1: POST /api/auth/register - Successfully creating a new USER account.

# 6 Conclusion

Through Lab 07, I successfully implemented a robust Fullstack Role-Based Access Control system. Key takeaways include:

- Enforcing server-side security using Express middleware.
- Securing REST APIs with stateless JWT authentication.
- Implementing role-based conditional rendering in a React frontend.
- Practicing professional documentation standards for fullstack development.

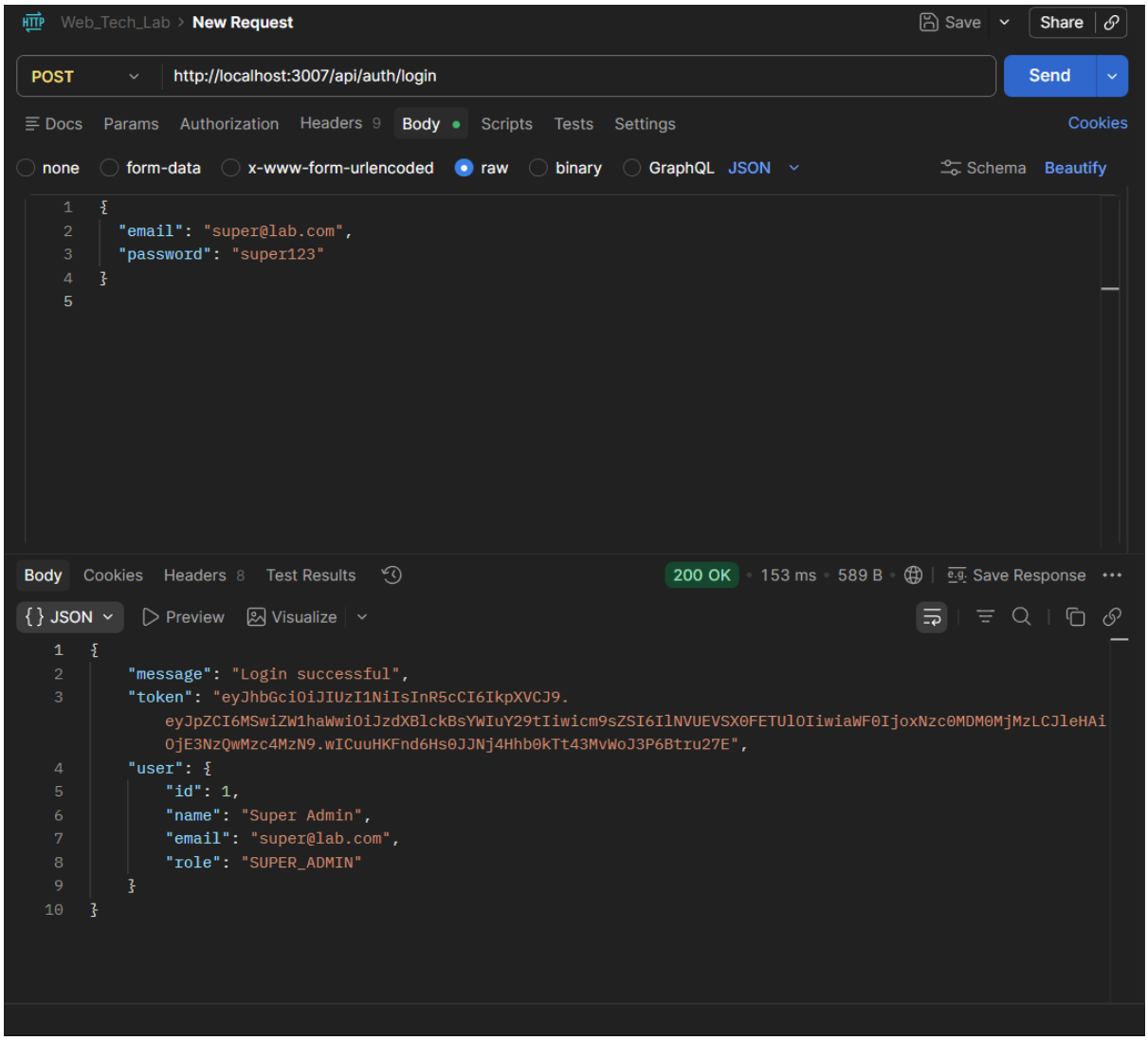


Figure 2: Fig-2: POST /api/auth/login - Obtaining JWT token and user profile for Super Admin.

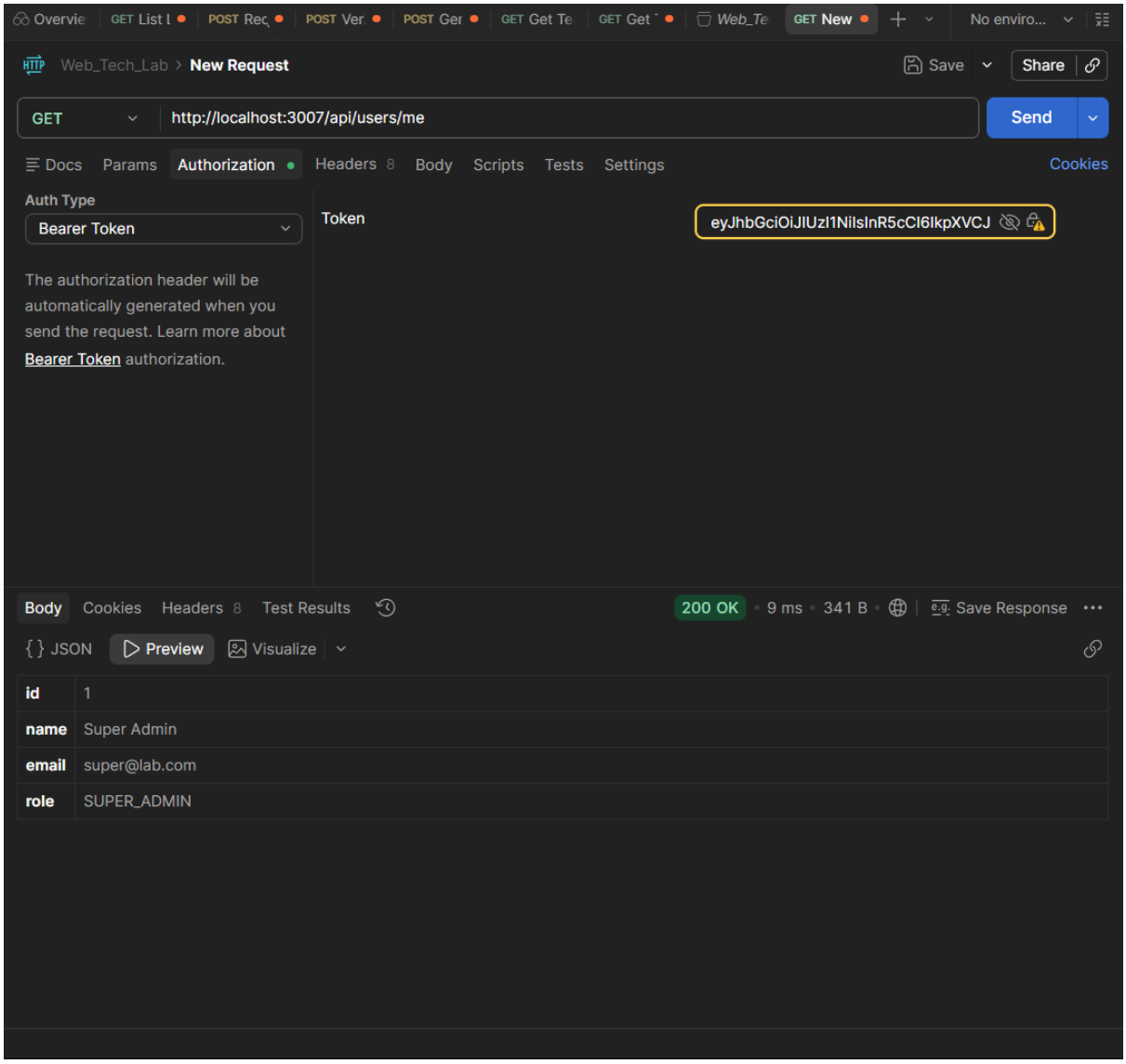


Figure 3: Fig-3: GET /api/users/me - Accessing own profile data using Bearer token.

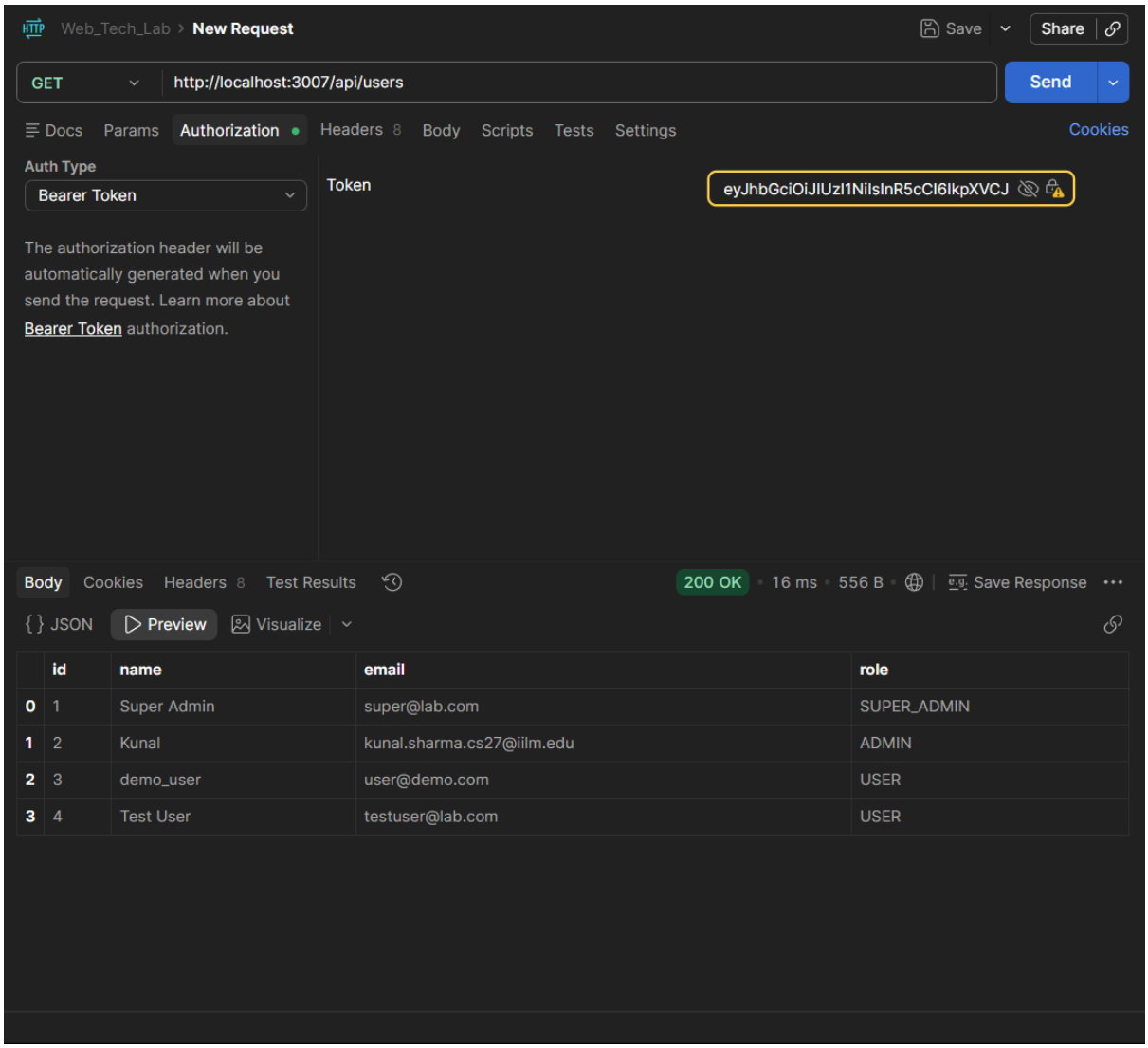


Figure 4: Fig-4: GET /api/users - SUPER_ADMIN viewing the complete user list.

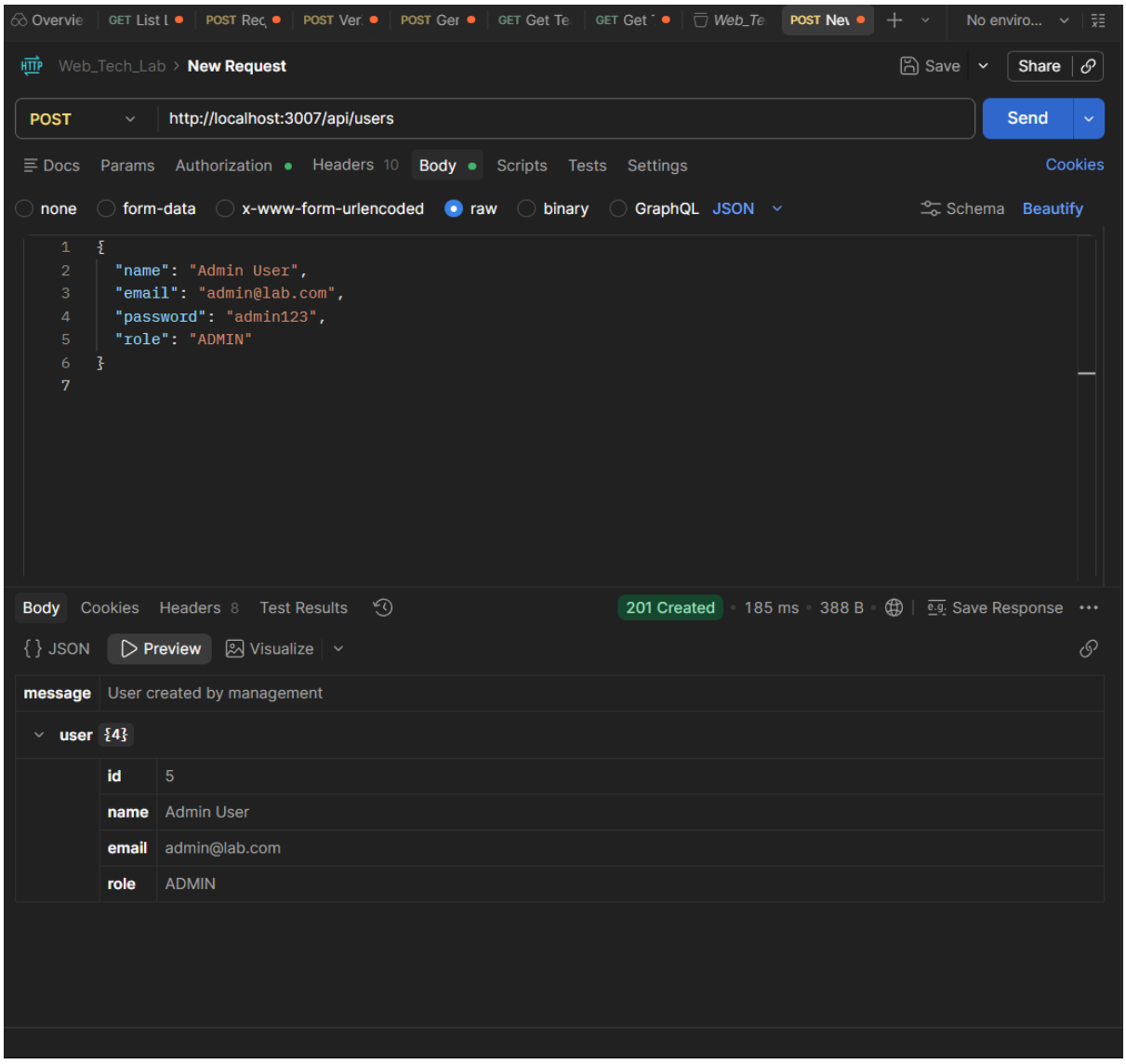


Figure 5: Fig-5: POST /api/users - SUPER_ADMIN creating an ADMIN account.

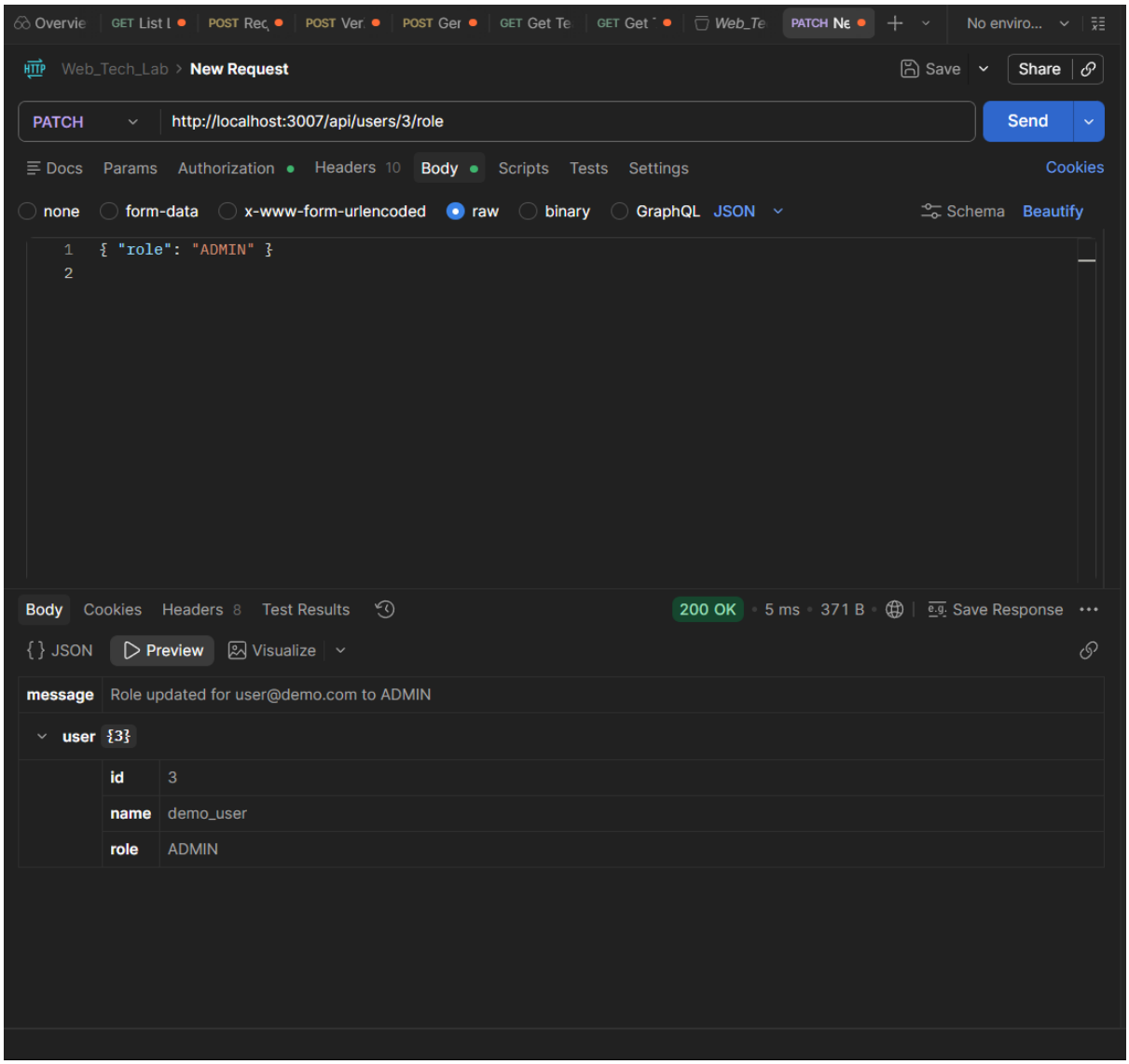


Figure 6: Fig-6: PATCH /api/users/:id/role - SUPER_ADMIN updating a user's role.

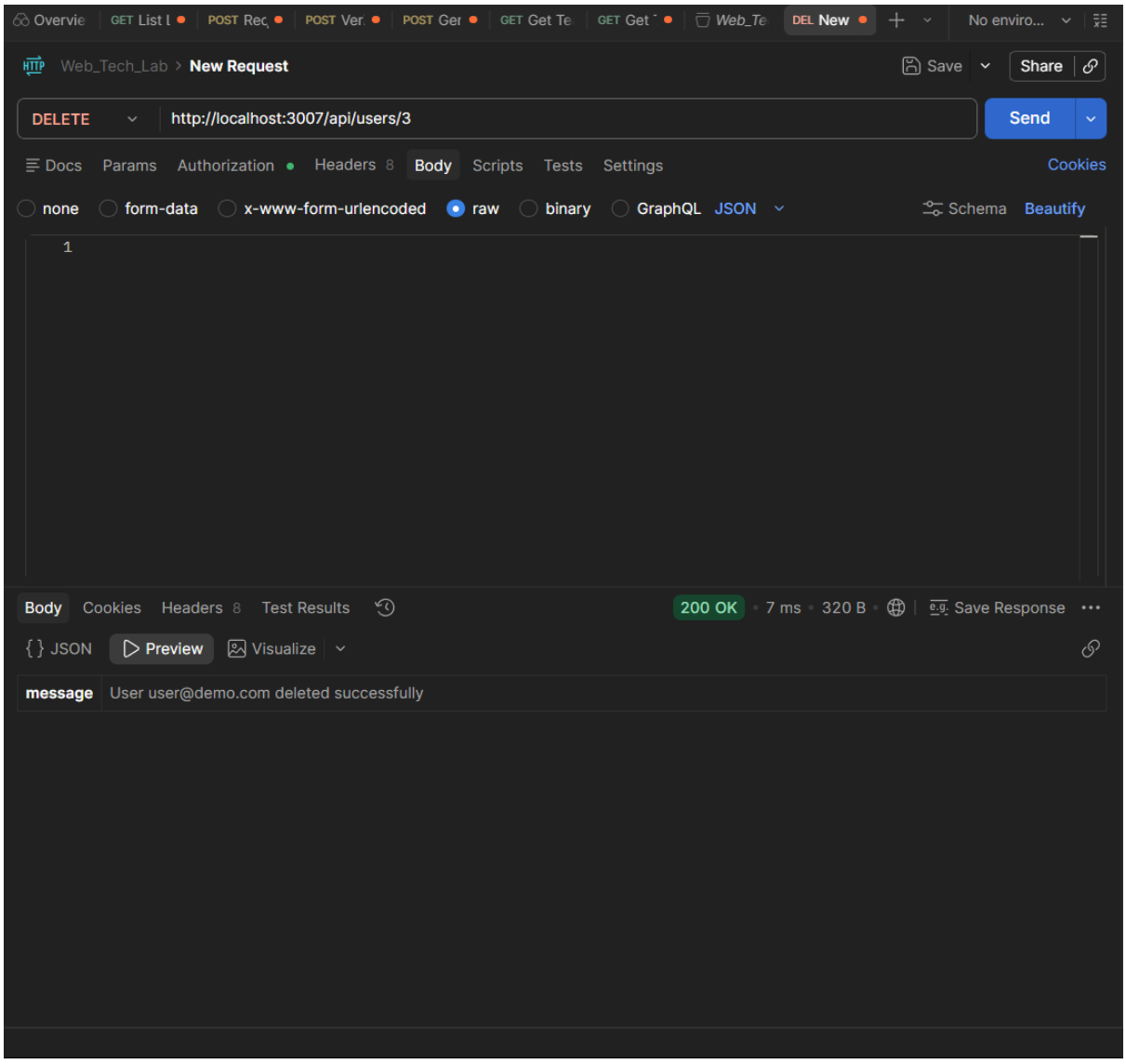


Figure 7: Fig-7: DELETE /api/users/:id - SUPER_ADMIN deleting a user account.

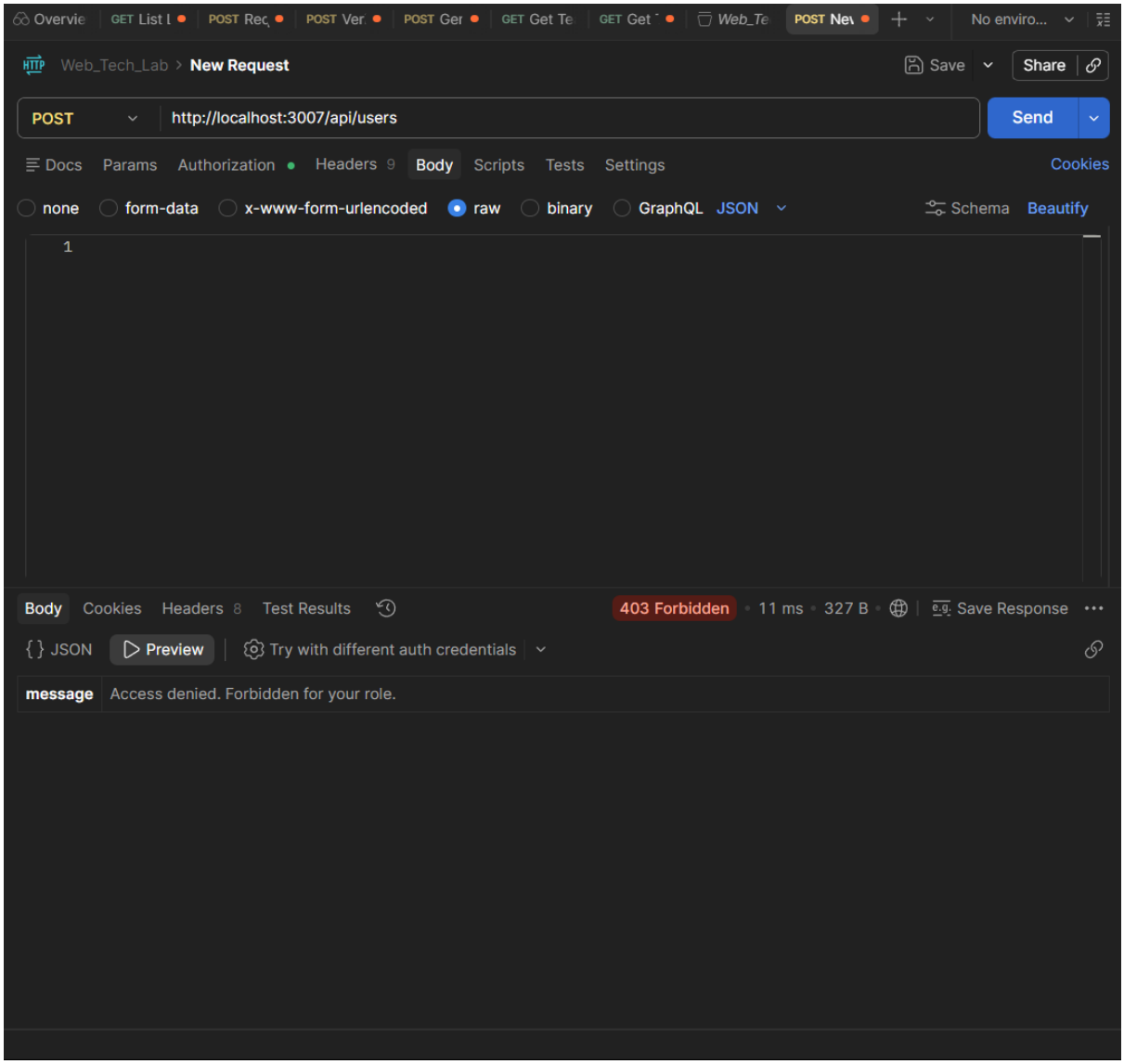


Figure 8: Fig-8: GET /api/users by USER role returns 403 Forbidden (RBAC enforced).

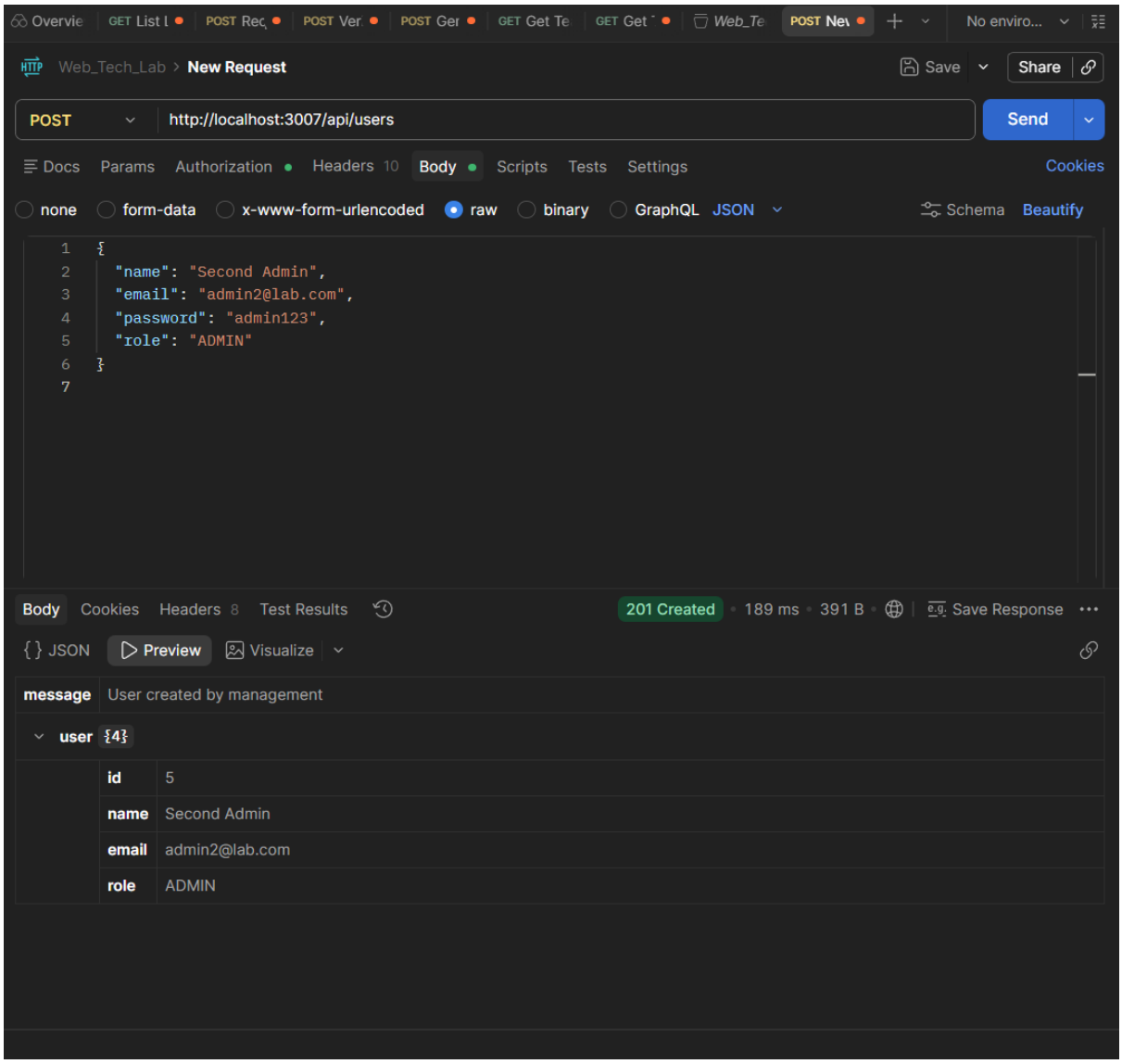


Figure 9: Fig-9: POST /api/users by ADMIN role trying to create another ADMIN returns 403 Forbidden.

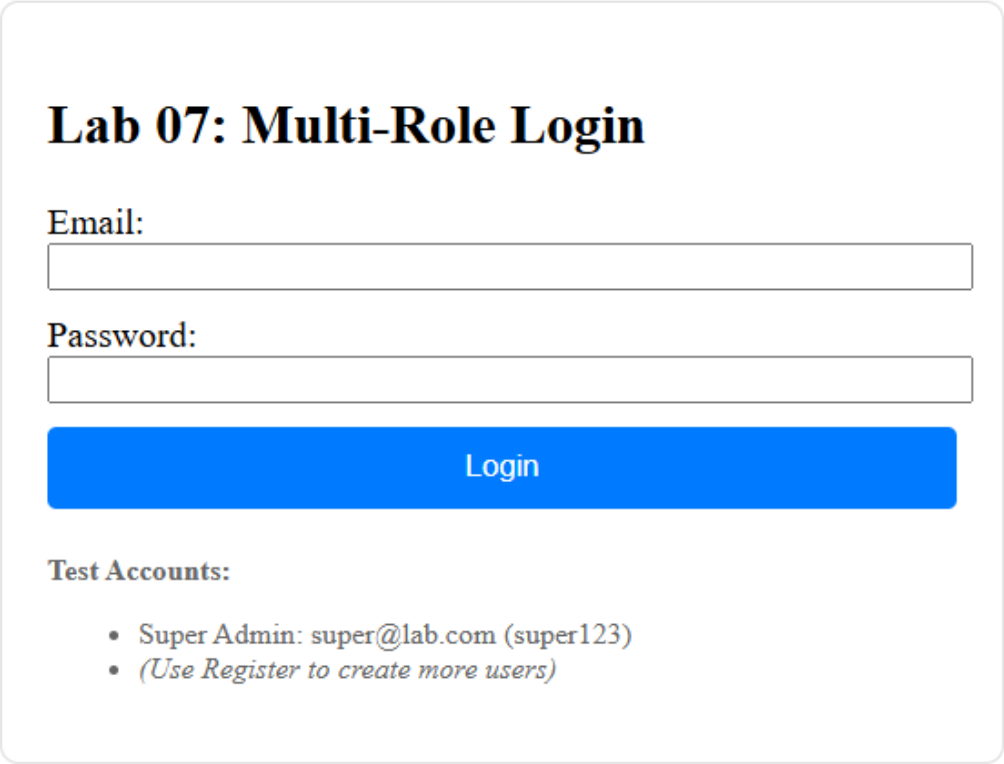


Figure 10: Fig-10: Login Page - Interface with test account references.

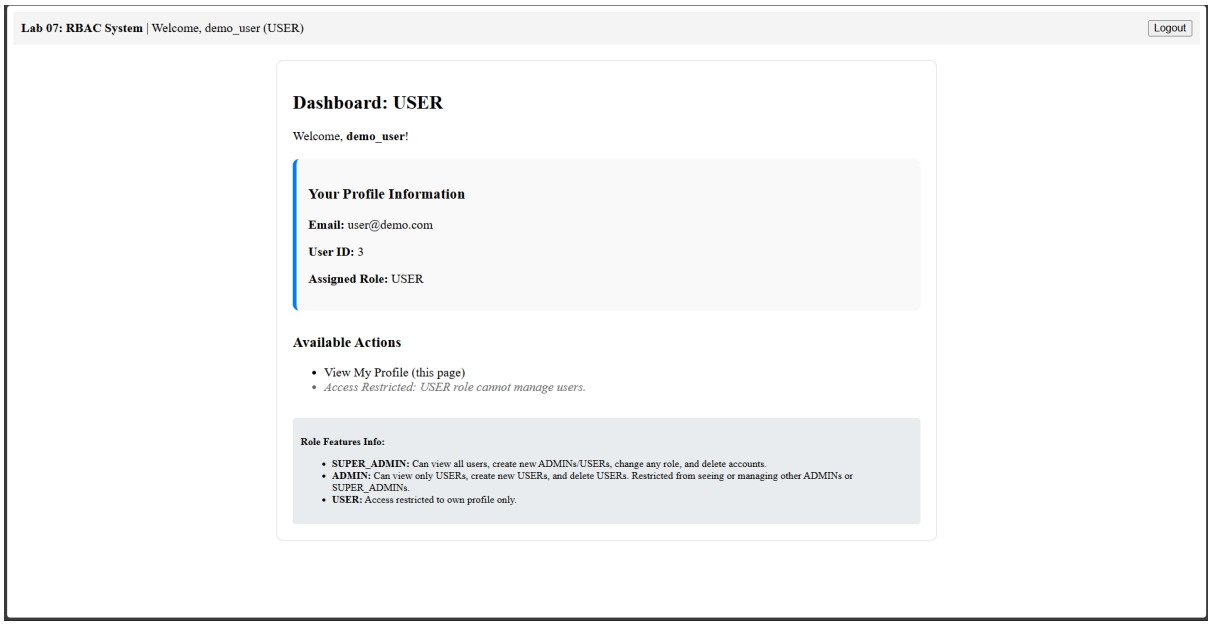


Figure 11: Fig-11: USER Dashboard - Restricted view showing only profile info.

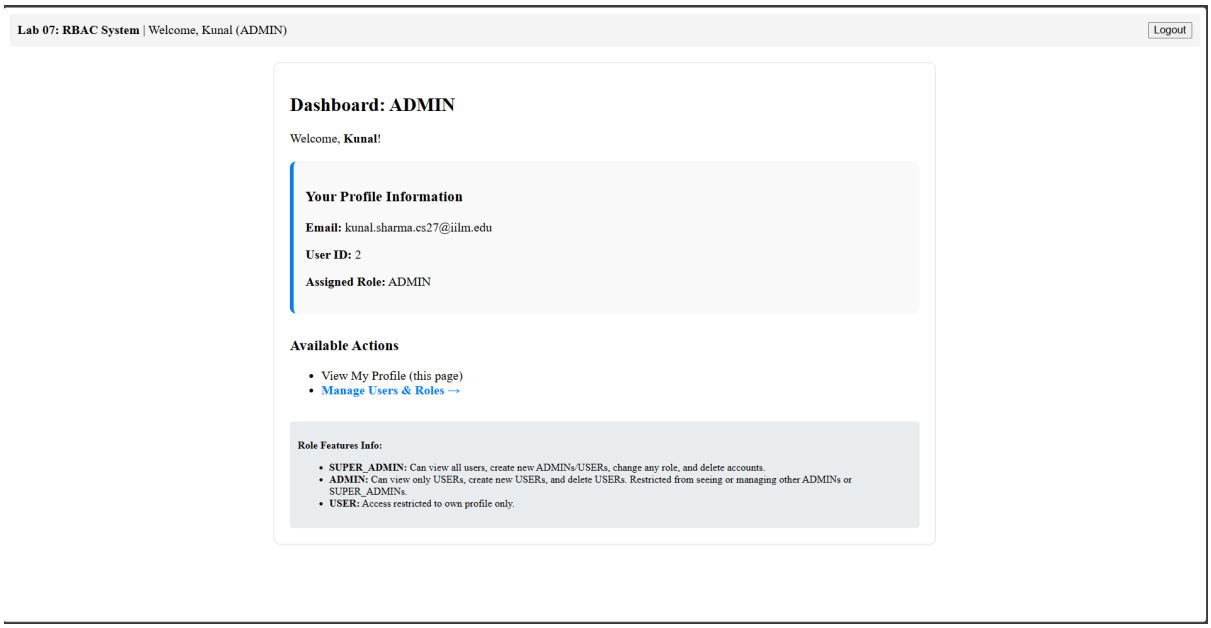


Figure 12: Fig-12: ADMIN Dashboard - View restricted to USER accounts only.

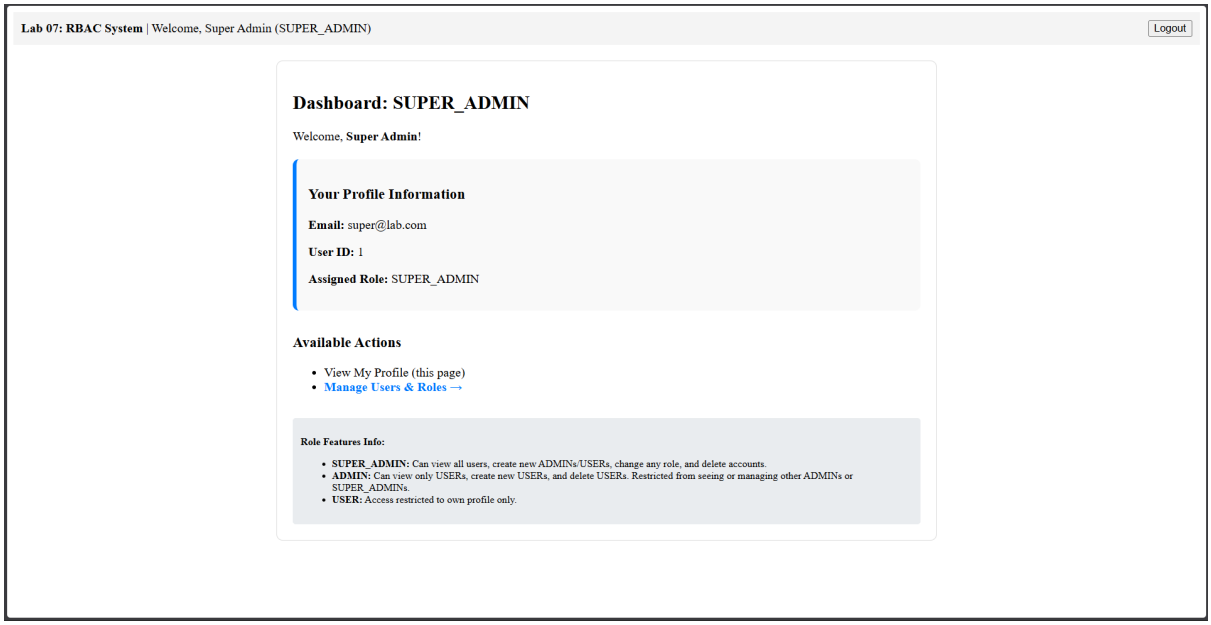


Figure 13: Fig-13: SUPER_ADMIN Dashboard - Full user management with role controls.